

Validating What You Built

Diagnostics for Regression, Classification, Trees & Clustering — Session 09/01/2026

- Every model gets fit, scored on a holdout set, and evaluated against a threshold
- Four models from last session, one protocol, run twice — once in R, once as database aggregate functions
- One model breaks on purpose, so a diagnostic can catch it and we fix it live

Why This Isn't Optional at Scale

What Breaks in Production

- A revenue model quietly 20% off skews marketing spend across hundreds of cities
- A delivery-availability classifier that looks accurate in a notebook fails on restaurants it's never seen
- The cost shows up weeks later — in a P&L or a support queue — not in the training run

The Protocol, One Line

- Fit it, score it on a holdout set the model never trained on
- Evaluate the result against a stated threshold — a metric with no threshold is an opinion
- Run the same evaluate step a second way, as database aggregate functions

One Dataset, One Holdout Split

The Running Dataset

- Same self-generated Zomato/Swiggy restaurant dataset as last session (set.seed(42), 200 restaurants)
- platform, cost, distance, rating, votes — with revenue, delivery, and price tier derived from them
- Zero external files — runs identically on any machine

The 70/30 Holdout Split

- 70% of restaurants train each model; the other 30% never touch training at all
- Every metric on the slides that follow is measured on that untouched 30%
- A model only gets credit for what it does on restaurants it has never seen

Linear Regression

Fit & Score

- Fit on distance from city center and vote count against estimated revenue
- Scored on the 30% holdout set — restaurants the model never trained on

Metric	Value	Threshold	Status
Holdout R ²	0.84	≥ 0.60	Within threshold
Holdout RMSE (% of mean revenue)	~17%	≤ 20%	Within threshold

Linear Regression — Reading the Panel

Beyond the Headline Number

- Residual plot (actual – predicted, plotted against predicted) shows no funnel shape and no curve
- Errors scatter randomly around zero across the whole prediction range
- That randomness is what 'the model isn't missing a pattern' looks like — a curved or funnel-shaped residual plot would mean the model is systematically wrong somewhere specific
- R^2 alone can't tell you that; you have to look at the residuals

Logistic Regression

Fit & Score

- Fit on average cost and rating against has_online_delivery
- Scored on the same 30% holdout set

Metric	Value	Threshold	Status
Holdout AUC	0.77	≥ 0.70	Within threshold
Holdout accuracy	73%	$\geq 70\%$	Within threshold
Recall, "no delivery" class	~20%	$\geq 50\%$	Flagged

The Headline-Metric Trap

What the Confusion Matrix Shows

- Accuracy and AUC both clear a reasonable bar — that's exactly the trap a headline metric sets
- Of the restaurants that genuinely don't offer delivery, the model correctly flags only about 1 in 5
- It has learned to predict 'yes, delivery' by default, because most restaurants in this data do offer it
- A delivery-fleet staffing decision built on this model would badly under-plan for the offline-only segment — a real operational miss accuracy and AUC alone never surface

A Quick Definition: "Cross-Validated" Error

Split training data into k folds $\rightarrow$ train on $k-1$, validate on the 1 held out $\rightarrow$ rotate $\rightarrow$ average

That average is the "cross-validated error" a tree reports — computed on training data alone

Not the Same Thing as the Holdout

- k-fold cross-validation happens entirely inside the training 70% — the 30% holdout is never touched
- rpart repeats this automatically at every candidate tree size while it grows, recording the result as xerror in its cptable
- It's a second, independent overfitting check — done before the holdout evaluation on the next slide
- Two different words, two different slices of data: don't read 'cross-validated error' as 'holdout error'

Decision Tree — Breaking It on Purpose

Training accuracy: 100% | Holdout accuracy: 40%

A 60-point gap is the signature of memorizing training data

What Happened

- Tree grown with no depth limit ($cp = 0$, $minsplit = 2$) on rating, votes, platform, distance
- 69 splits — rich enough to fit noise in the training restaurants perfectly
- rpart's own cross-validation table (built while the tree grows) already flags this: cross-validated error is worst at the tree's full, unpruned size

Decision Tree — The Fix

Metric	Value	Threshold	Status
Unpruned: train – holdout gap	~60 pts	≤ 15 pts	Flagged
Pruned: train – holdout gap	~6 pts	≤ 15 pts	Within threshold

The Honest Lesson

- Pruned tree collapses to 1 split; train (~48%) and holdout (~42%) now land close together
- Absolute accuracy is still underwhelming — and that's the real finding, not a failure of pruning
- Pruning fixed the overfitting; it also revealed these features don't carry much price-tier signal
- Sometimes the fix isn't a fancier model — it's better features

k-Means Clustering

No Holdout Labels Here

- Clustering has no 'correct' cluster to score against — the confirmation is trying a range of k
- Fit on scaled average cost and rating; k = 2 through 5 compared by silhouette score

Metric	Value	Threshold	Status
Silhouette, best k (k=3)	0.42	≥ 0.50	Flagged

k-Means — Reading the Panel

What 0.42 Means Here

- $k = 3$ is the best of the four values tried — but 0.42 sits below the 0.50 bar for 'clearly separated'
- Cost and rating alone produce segments that overlap more than they're distinct
- Useful to know before building a marketing plan around three crisp restaurant personas
- Treat the cluster boundaries as a starting point for judgment, not a hard rule

The Same Evaluate Step, a Second Engine

group_by() + summarise() == the grammar dbplyr pushes straight to SQL

Same protocol, no code changes to run it against a live table

Why This Matters in Production

- Everything on the last 9 slides ran in R, in memory — every row pulled in first
- At platform scale, that evaluate step usually runs as an aggregate query against the database instead
- The grammar (group, then summarize) is identical either way — only where it executes changes

The Actual Push-Down

```
con <- DBI::dbConnect(RMySQL::MySQL(), dbname = "bda_pop", host = Sys.getenv("DB_HOST"))
restaurants_db <- tbl(con, "restaurants")

restaurants_db %>%
  group_by(platform) %>%
  summarise(avg_revenue = mean(estimated_revenue),
            delivery_rate = mean(has_online_delivery)) %>%
  show_query()    # prints the SELECT ... GROUP BY — nothing pulled into R yet
  collect()      # only this line crosses the wire and returns rows
```

Credentials from an env var, per this environment's own no-hardcoded-secrets rule.

The Point of `show_query()`

- Same `group_by()/summarise()` verbs from the last slide — dbplyr compiles them to real SQL, it doesn't run them row-by-row in R

SAME NUMBERS, GROUPED BY PLATFORM AND TIER

Second Engine — The Results

Platform	Restaurants	Avg. Revenue	Avg. Abs. Error	Delivery Rate
Swiggy	111	1,642	242	71%
Zomato	89	1,776	277	72%

Price Tier	Restaurants	Delivery Rate	Avg. Rating
Budget	25	40%	3.08
Moderate	31	52%	3.45
Premium	77	75%	3.06
Luxury	67	88%	3.36

SUMMARY

Every Model, One Glance

Model	Metric	Value	Threshold	Status
Linear Regression	Holdout R ²	0.84	≥ 0.60	Within threshold
Linear Regression	Holdout RMSE (% of mean)	~17%	≤ 20%	Within threshold
Logistic Regression	Holdout AUC	0.77	≥ 0.70	Within threshold
Logistic Regression	Recall, "no delivery"	~20%	≥ 50%	Flagged
Decision Tree (unpruned)	Train – holdout gap	~60 pts	≤ 15 pts	Flagged
Decision Tree (pruned)	Train – holdout gap	~6 pts	≤ 15 pts	Within threshold
k-Means (k=3)	Avg. silhouette	0.42	≥ 0.50	Flagged

TAKEAWAYS

What to Bring Back to Your Own Models

- Never trust a metric with no threshold — decide the bar before you look at the number
- Always score on a holdout set, or on cross-validated error the model never optimized against directly
- Read the component metrics, not just the headline one — the confusion matrix caught what accuracy hid
- A flagged diagnostic isn't always a model problem — sometimes it's telling you to get better features
- Whatever you can compute in R, ask whether it could run as a database aggregate query instead

Six Sessions, One Lifecycle

Date	Lifecycle Phase	What We Built
07/25	All 5 phases, overview	Telco churn workshop — prep through operationalizing, end to end
08/01	Data Prep + Initial EDA	Distributions, initial exploration
08/11	Statistical foundations	Summary tests, sound vs. significant
08/18	Model Planning	H0/H1, technique selection by business objective
08/25	Model Building	ML/DL algorithm survey, technical foundations
09/01 (Today)	Validation	Diagnostics — R holdout checks, then database aggregate functions

Where the Loop Closes

- Validation is the gate: only a model that clears today's checks is fit to reach the Communicating and Operationalizing phases from 07/25 — that's why it comes last, not first

WHERE THIS GOES FROM HERE

Next Up

Questions & What's Next

- Every model we validated today feeds directly into how results get communicated and operationalized
- Bring a model of your own next session — we'll run this same fit/score/evaluate protocol on it live
- Questions?